

Machine Learning in Practice I

Lecture 11

Antal Jakovac, 2025

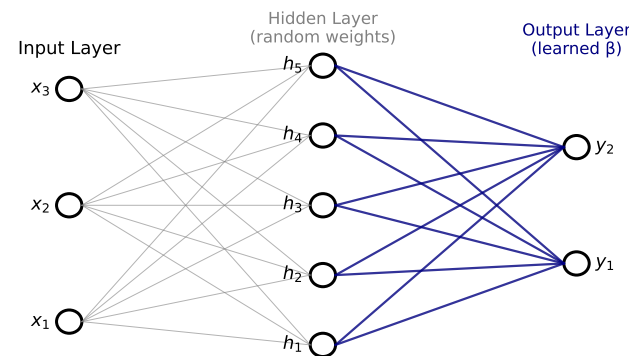
Extreme Learning Machine



- what is the best basis to represent the output?
 - perceptron, linear SVM → linear basis $x \rightarrow Mx + b$
 - nonlinear SVM → arbitrary fixed kernel $x \rightarrow \sum_n \alpha_n y_n K(x_i, x; \Theta) + b$
- DNNs: multilayer structure → preceding layers + last layer
 - preceding layers → generate the basis, fitted to the problem
 - last layer → use linear combination of basis elements
- Extreme Learning Machine (ELM)
 - use a random basis → technically a layer with fixed weights

Extreme Learning Machine

- data in matrix representation: $X \in \mathbb{R}^{N \times N_x}$, $Y \in \mathbb{R}^{N \times N_y}$
- define an intermediate basis: $g(X) = \sigma(X \cdot A + b) \in \mathbb{R}^{N \times M}$
 - σ is a pointwise non-linear function (e.g. *tanh*, or Gaussian)
 - $A \in \mathbb{R}^{N_x \times M}$, $b \in \mathbb{R}^M$ fixed parameters
- expand Y in this basis: $y = g(x) \cdot \beta$
 - $\beta \in \mathbb{R}^{M \times N_y}$ tunable parameters
 - best value from minimizing the chi-squared function (use regularization!)



$$\chi^2(\beta) = (g(X) \cdot \beta - Y)^T C^{-1} (g(X) \cdot \beta - Y) \Rightarrow \beta = (g^T C^1 g)^{-1} g^T C^{-1} Y$$

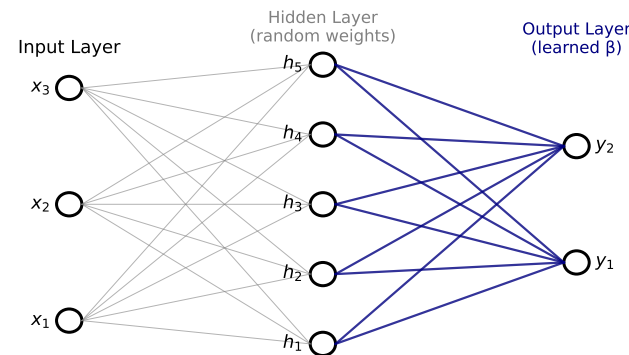
- simple package for ELM

```
# ELM package
import numpy as np

class ELM:
    def __init__(self, n_hidden=50, activation=np.tanh):
        self.n_hidden = n_hidden
        self.activation = activation

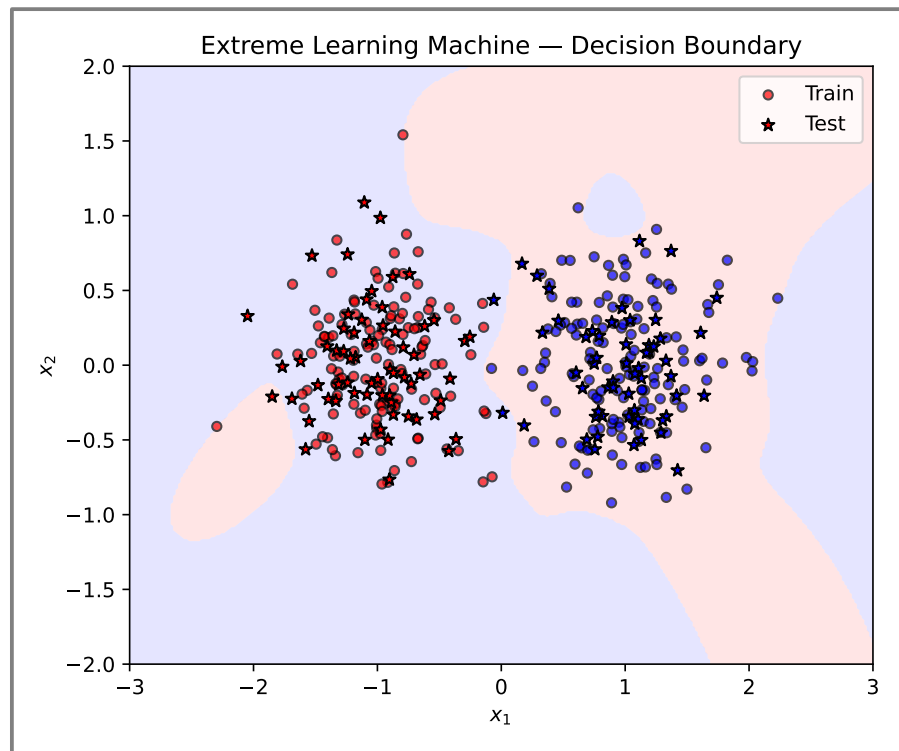
    def fit(self, X, y):
        self.W = np.random.randn(X.shape[1], self.n_hidden)
        self.b = np.random.randn(self.n_hidden)
        H = self.activation(X @ self.W + self.b)
        self.beta = np.linalg.pinv(H) @ y

    def predict(self, X):
        H = self.activation(X @ self.W + self.b)
        return H @ self.beta
```

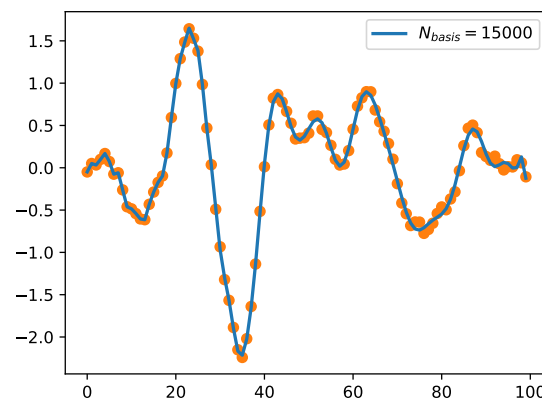
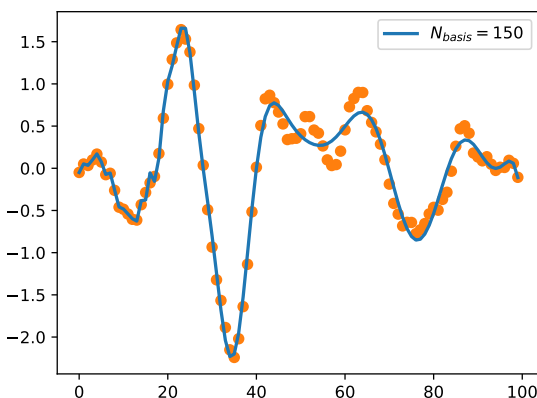
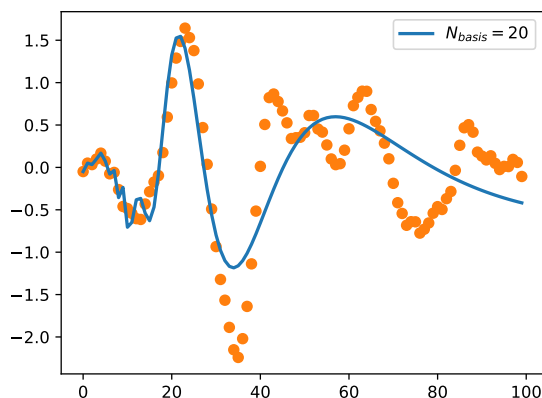


Extreme Learning Machine

- example dataset
- activation function: $\tanh$
- accuracy: 97%



- function fitting with ELM
- random function, and different hidden layer dimensions
- basis not too effective to describe highly oscillating functions



Gaussian representation of data distribution

Gaussian Mixture Models

- interconnected data are often sit in a compact region → ellipsoid
- find the ellipsoid containing most of the data! $p(x) \sim \prod_{n=1}^N e^{-\frac{1}{2}(x_n - \mu)C^{-1}(x_n - \mu)}$
$$L(\mu, C) = p(x | \mu, C) \text{ maximum} \Rightarrow \mu \approx \frac{1}{N} \sum_{n=1}^N x_n, \quad C_{ij} \approx \frac{1}{N-1} \sum_{n=1}^N (x_n - \bar{x})_i (x_n - \bar{x})_j$$
- probability level → “distance” $\chi^2 = (x_n - \mu)C^{-1}(x_n - \mu)$
- equidistance surfaces $\chi^2 = \text{constant} \rightarrow$ ellipsoids
- generate points: $x = \mu + r \sum_{a=1}^{\dim} \alpha_a \sqrt{\lambda_a} v_a \rightarrow \sum_{a=1}^{\dim} \alpha_a^2 = 1, \quad C v_a = \lambda_a v_a$

Gaussian Mixture Models

- technical slide: equipotential surfaces from correlation matrix
- we want to satisfy $\chi^2 = (x - \mu) C^{-1} (x - \mu) = r^2$
- eigensystem $C v_a = \lambda_a v_a$, eigenvectors orthogonal
- ansatz: $x = \mu + r \sum_{a=1}^{\dim} \alpha_a \sqrt{\lambda_a} v_a \Rightarrow x - \mu = r \sum_{a=1}^{\dim} \alpha_a \sqrt{\lambda_a} v_a$
- verify: $\chi^2 = r^2 \sum_{a,b=1}^{\dim} (\alpha_a \sqrt{\lambda_a}) (\alpha_b \sqrt{\lambda_b}) v_a C^{-1} v_b = r^2 \sum_{a,b=1}^{\dim} (\alpha_a \sqrt{\lambda_a}) (\alpha_b \sqrt{\lambda_b}) \lambda_a \delta_{ab} = r^2$

- technical slide: generate data with given μ, C

- data with normal distribution: ξ

- transform it $\eta = \sqrt{C} \xi + \mu$

- check:

$$E(\eta) = \mu$$

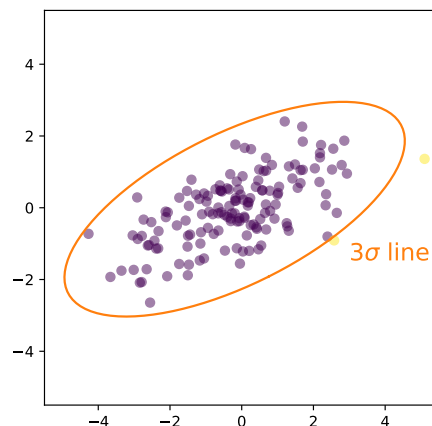
$$E((\eta - \mu) \otimes (\eta - \mu)) = \sqrt{C} E(\xi \otimes \xi) \sqrt{C} = C$$

- for 2D data we can directly specify the direction: $ev = \begin{pmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{pmatrix}$

```
# generating Gaussian with given mu and C  
def generate_Gaussian(mu, C, Ndata):  
    dim=mu.shape[0]  
    xi = np.random.normal(0,1, (dim, Ndata))  
    lam,ev =np.linalg.eigh(C)  
    sqrt_C = ev@np.diag(np.sqrt(lam))@ev.T  
    return sqrt_C@xi + mu
```

Gaussian Mixture Models

- example: 2D points $C = \begin{pmatrix} 2.5 & 1 \\ 1 & 1 \end{pmatrix} \Rightarrow \lambda = (0.5, 3), \text{ ev} = \frac{1}{\sqrt{5}} \begin{pmatrix} 1 & -2 \\ 2 & 1 \end{pmatrix}$
 - generate points
 - numerically determine correlation matrix and eigensystem
 - draw ellipses containing 99% of data → about 3σ in 2D



Gaussian Mixture Models

- if we have several classes → Naive Bayes models
 - assume that each class are independent and points lie in an ellipsoid → can be represented by Gaussian distribution
 - determine the probability to belong to each classes separately

```
# Naive Bayes classification
```

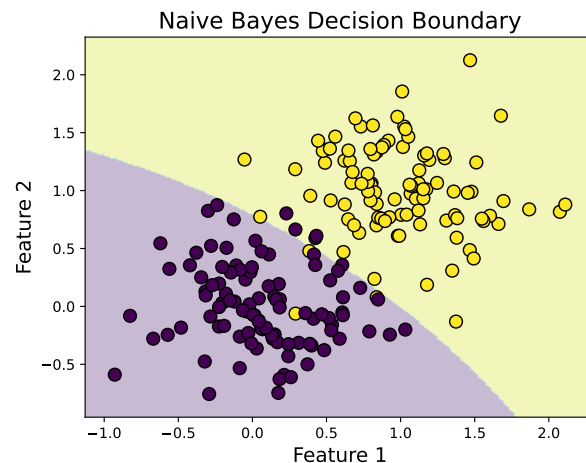
```
from sklearn.naive_bayes import GaussianNB
```

```
nb = GaussianNB()
```

```
nb.fit(X_train, y_train)
```

```
y_pred = nb.predict(X_test)
```

```
→ accuracy: 96%
```



- unsupervised generalization → GMM
 - assume the probability distribution is a **sum** of Gaussian terms
 - fix the number of clusters → *do not provide labels (unsupervised)*
 - find the best parameters by maximizing the likelihood

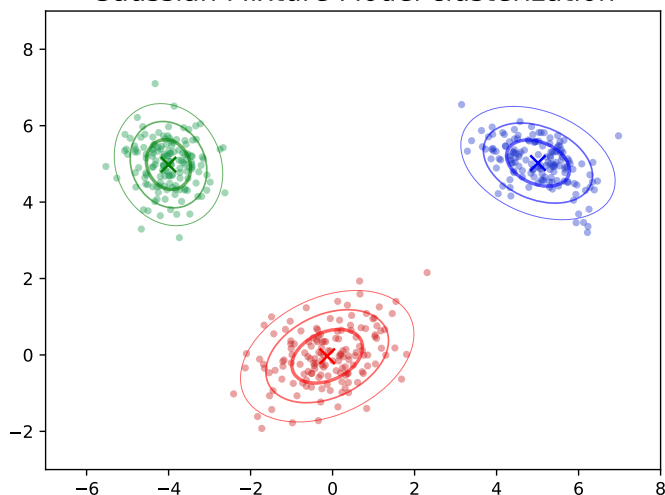
$$L(\pi, \mu, C | x) = p(x | \pi, \mu, C) = \sum_{k=1}^K \pi_k N(x; \mu_k, C_k)$$

- we need convex combinations $\pi_k \in [0, 1], \sum_{k=1}^K \pi_k = 1$
- use Expectation–Maximization (EM) algorithm

Gaussian Mixture Models

- example: clustering in 2D

Gaussian Mixture Model clusterization

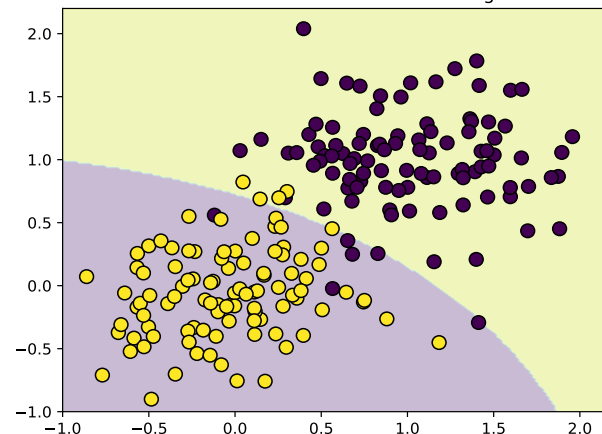


```
# Gaussian Mixture Models
```

```
from sklearn.mixture import GaussianMixture
```

```
gmm = GaussianMixture(n_components=2, covariance_type='full',  
random_state=0, n_init=10)gmm.fit(X)
```

Gaussian Mixture Models decision regions



- technical slide: the Expectation–Maximization (EM) algorithm

- start from an initial guess π, μ, C

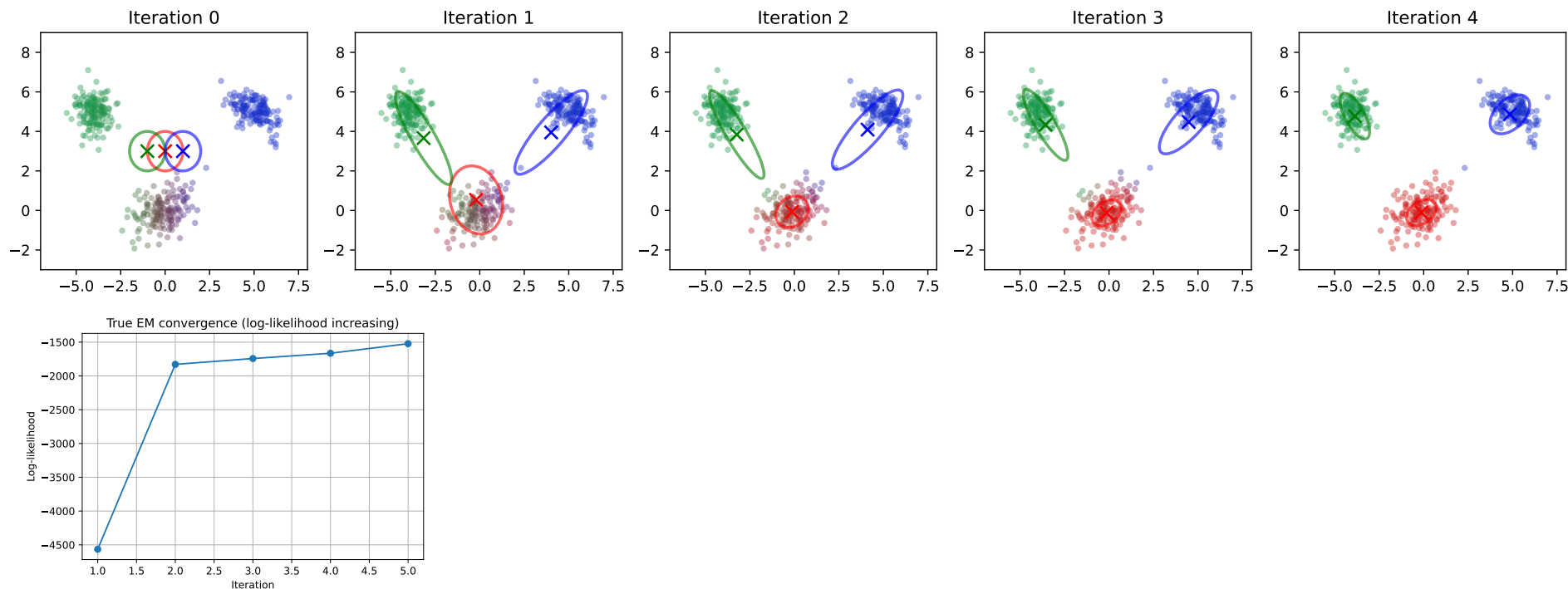
- compute “responsibilities”
(closest component weight) $r_{ik} = \frac{\pi_k N(x_i; \mu_k, C_k)}{\sum_j \pi_j N(x_i; \mu_j, C_j)}$

- redefine parameters
(attract to closest cluster)

$$N_k = \sum_i r_{ik}, \quad \pi_k = \frac{N_k}{N}, \quad \mu_k = \frac{1}{N_k} \sum_i x_i r_{ik}, \quad C_k = \frac{1}{N_k} \sum_i (x_i - \mu_k)(x_i - \mu_k)^T$$

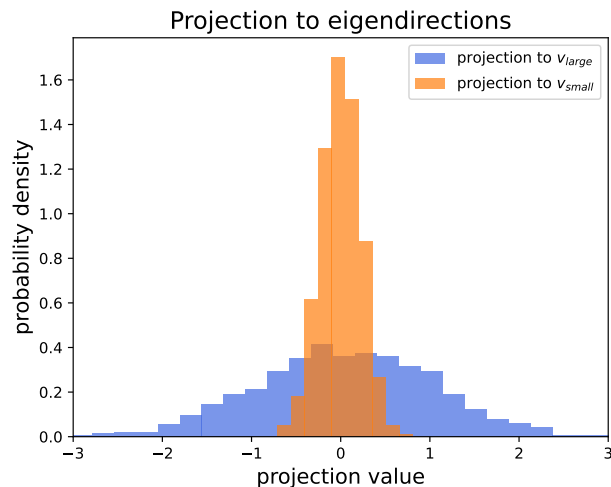
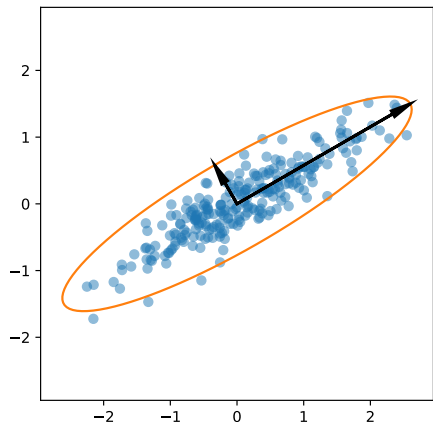
- repeat until converges

- technical slide: the Expectation–Maximization (EM) algorithm



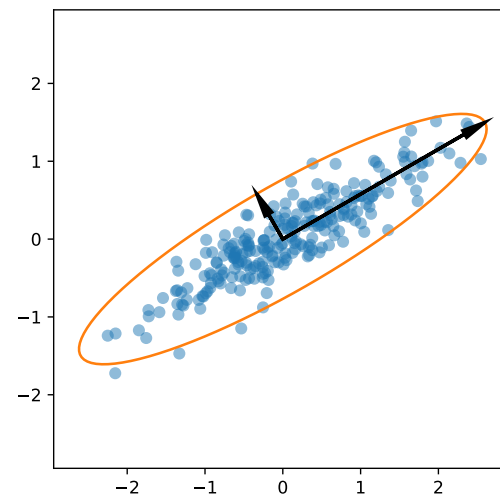
Principal Component Analysis (PCA)

- choose an orthonormal coordinate system aligned with the data!
- we fitted an ellipsoid $\rightarrow$ find the eigenvectors
 - $\rightarrow$ smallest eigenvalues: data vary the least
 - $\rightarrow$ largest eigenvalues: data vary the most



Principal Component Analysis (PCA)

- PCA: keep only largest eigenvalues
 - ➔ compression → descriptive features
 - ➔ noise reduction
 - ➔ representation of the largest changes
- small eigenvalues:
 - ➔ approximately constant for all data in a class
 - ➔ classification → selective features
 - ➔ laws



- technical slide: compression with PCA

$$\text{data} = \{x_n \in \mathbb{R}^d \mid n=1, 2, \dots, N\} \quad X_{ni} = (x_n)_i$$

$$\mu_i = \frac{1}{N} \sum_n X_{ni}, \quad \bar{X}_{ni} = X_{ni} - \mu_i \quad C = \bar{X}^T \bar{X}$$

$$C v_a = \lambda_a v_a, \quad |v_a| = 1 \Rightarrow E((\bar{X} \cdot v_a)^2) = E(v_a^T \bar{X}^T \bar{X} v_a) = E(v_a C v_a) = \lambda_a$$

- eigenvalue = projection value squared
- PCA values: $Y_{na} = v_a^T \bar{X}_n$ for the largest eigenvalues $\rightarrow$ fewer data

Principal Component Analysis (PCA)

- technical slide: data reconstruction with PCA
- most information is in the subspace of largest eigenvalues

$$\bar{x}_n = \sum_{a=1}^d v_a (v_a^T \cdot \bar{x}_n) \approx \sum_{\lambda_a / \lambda_{\max} > r_{\min}} v_a (v_a^T \bar{x}_n) = \sum_{\lambda_a / \lambda_{\max} > r_{\min}} Y_{na} v_a$$

- therefore $x_n = \sum_{\lambda_a / \lambda_{\max} > r_{\min}} Y_{na} v_a + \mu_i$

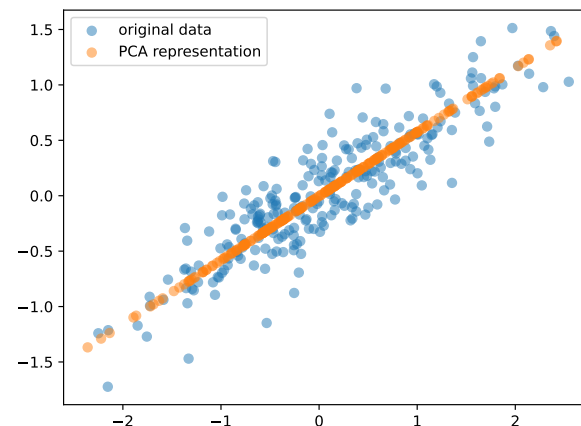
PCA representation of data

from sklearn.decomposition import PCA

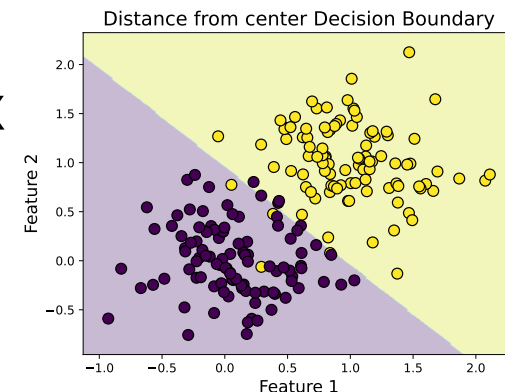
```
pca = PCA(n_components=1)
```

```
data_pca=pca.fit_transform(data.T)
```

```
data_reconstructed=pca.inverse_transform(data_pca)
```



- classification with laws $\rightarrow$ LLT
- smallest eigenvectors of the PCA matrix $\rightarrow$ laws
- classification features: $f_a = (X - \mu_{class}) \cdot v_{class,a}$, $\lambda_a < r \lambda_{max}$
 - $\rightarrow$ if $X \in \text{class} \Rightarrow E(|f_a|^2) = \lambda_a$ small value
 - $\rightarrow$ if $X \notin \text{class} \Rightarrow E(|f_a|^2) =$ usually a larger value
- so $\sum_a |f_a|^2$ is the smallest for the true class index
 - $\rightarrow$ if $r=1$: distance from the centre



END OF LECTURE 11